

Jacob Serfaty

Machine Learning for Computer Systems Final Project

Improving NetML Classification Results with Inter-Arrival Time Statistics

For my final project, I tested to see whether including inter-arrival time (IAT) statistics alongside the standard NetML STATS features would improve the results of supervised learning models. I hypothesized that IAT features could have a huge impact on training because they reveal information about the timing in which data is transferred. Many problems in networking correlate with packet timing, so predicting them only with information about packet size should, in theory, produce weaker results than predictions that are based on IAT statistics. For example, I hypothesized that [nPrint's Streaming Video Services](#) classification problem, which aims to determine which video service generated network information based solely on the first 10 SYN packets (flow creation) for each video session, is a problem where labels can be best distinguished using packet timing, so a model trained with IAT statistics should outperform a model that does not include these statistics.

To test my hypothesis, I trained 2 Random Forest Classifier models on this dataset: one model with only the standard NetML STATS features, and another model with both the NetML STATS features and IAT statistics (mean, median, standard deviation, min, max, lower quartile, upper quartile).

Preparing the data for this classification was surprisingly difficult. The data linked on the nPrint website was in the `.pcapng` format, which does not work natively with the standard NetML tools which expect `.pcap` files. Standard attempts to convert to a `.pcap` format did not work because the timestamps contained negative numbers. Also, the session IDs and streaming service labels were stored in per-packet comments, which are not components of `.pcap` files. As one final issue, despite the nPrint website claiming that “This dataset is filtered to contain only the first 10 SYN packets (flow creation) for each video session”, from what I could tell, the dataset was *not* already filtered. Though there were only TCP packets, there were far more than 10 packets for each video session (based on the session IDs). Also, as I was warned about in my project proposal, there wasn't a single instance I could find of 10 SYN packets in a row, as there

were SYN-ACK packets appearing instead. This makes sense as 10 SYN packets in a row for a single video session would likely indicate packet loss and network disturbance, but it is still something I have to deal with.

To convert the files, I opted to forgo `.pcap` files entirely and perform as much of the processing as I could using standard `.csv` files, as they are much easier and faster to work with. I used the Wireshark GUI to export the packet dissections into a `.csv` file, and then I used TShark to export the per-packet comments into another `.csv` file. I then loaded these files into Python and filtered to only take the first 10 SYN or SYN-ACK packets of each video session (where video sessions are determined by the session ID labels). I used both SYN and SYN-ACK packets together to ensure I had enough data, getting around the fact that there were never 10 SYN packets in a row for a given video session. This does introduce bi-directional processing time (since it includes both the client's message to the server and the server's acknowledgement response to the client), but I figured that this processing time could actually be a valuable data point for training. Overall, this data extraction gave me 10 packets for each of 20,856 video sessions.

I grouped packets into flows corresponding to their session IDs, so all 10 packets packets for each video session were included in the same flow, meaning each flow represents a single video session. Note that I excluded any video sessions with fewer than 10 packets to ensure consistency. I performed this flow grouping myself instead of relying on NetML, since NetML requires a `.pcap` file to determine flows and I did not have such a file. To trick NetML into understanding my flows, I had to create a custom `class` to represent a packet. To prevent egregious memory allocations and slow processing times, I optimized this `class` by using `__slots__` to prevent Python from creating an internal `__dict__` for each object and to instead store all packet information in a compact fixed-size array-like structure.

After grouping into flows, I used NetML to compute the IATs. In my understanding, NetML performed this computation by subtracting the start times of consecutive packets. This will necessarily include bidirectional traffic in the computation, but I believe these processing times are valuable features, so this is okay. Once I got the IATs, I used numpy to compute

statistics for the IATs. To finalize my feature preparation, I ran the NetML function to compute the standard NetML STATS features.

Using these per-flow features, I trained 2 Random Forest Classifier models with scikit-learn: One using only the NetML STATS features, and another using both the NetML STATS features and the IAT statistics. To evaluate both conditions fairly, I had to ensure that both conditions had the same train-test split. To do so, I used the same random seed when computing the train-test splits for both conditions. When training my models, I used Grid Search to optimize metaparameters including the number of estimators, the maximum depth, and the minimum samples at each split. Training took 50 seconds combined for both models.

After training, I computed statistical metrics about both models to compare their performance. All metrics are included below:

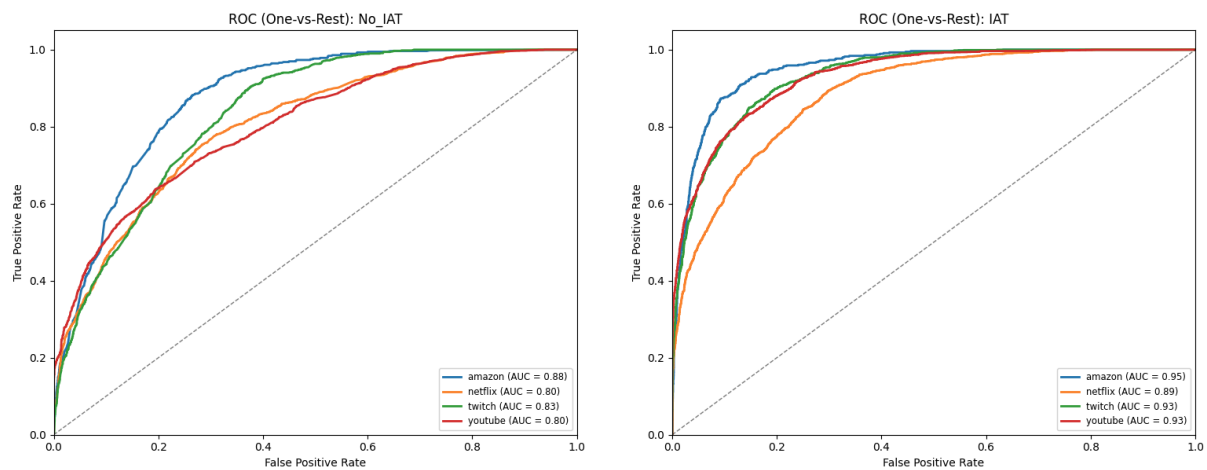
None

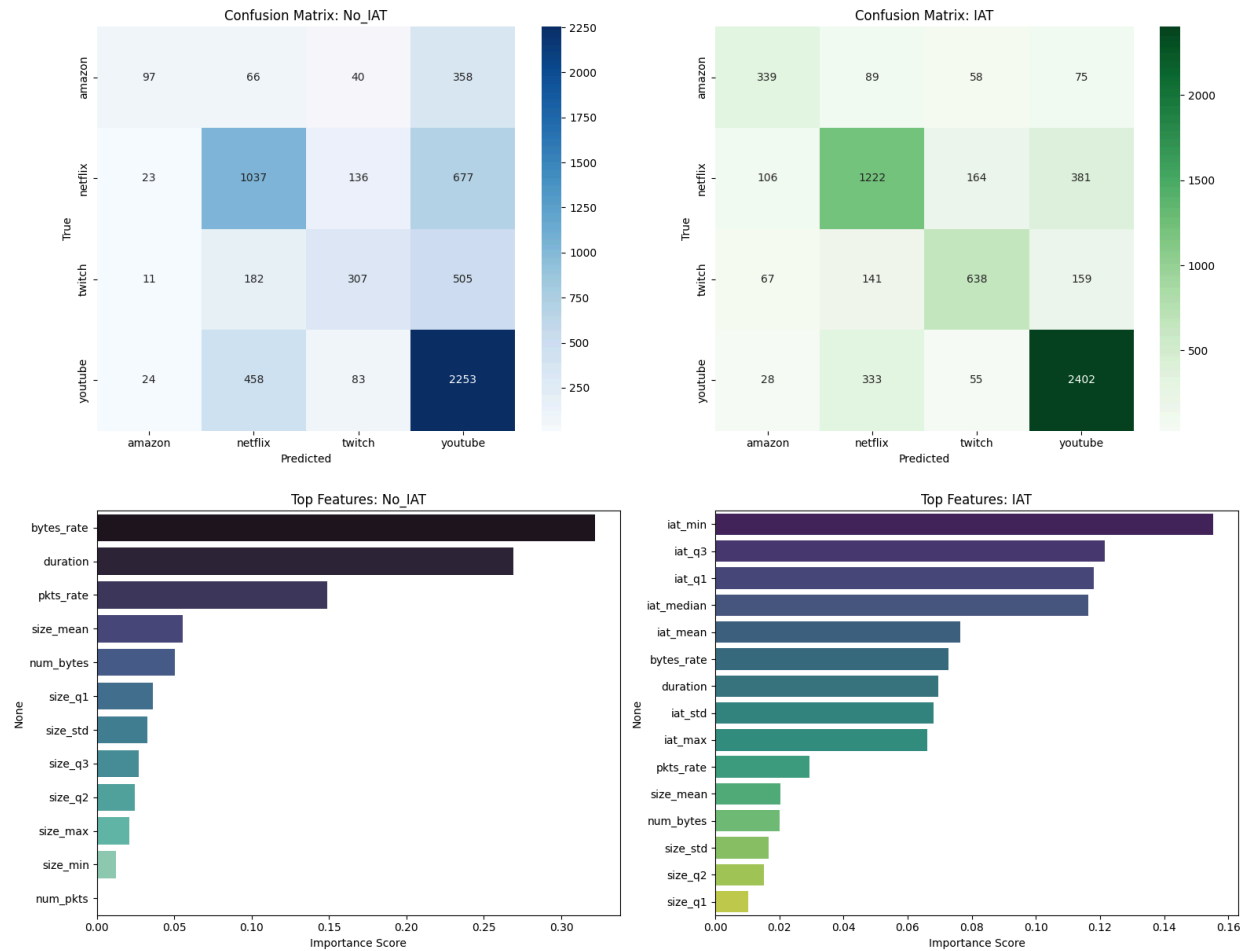
=====

FINAL METRICS COMPARISON

=====

	Model	Accuracy	Precision	Recall	F1 Score	AUC (OvR)	AUC (OvO)
0	No_IAT	0.5904	0.5888	0.5904	0.5657	0.8157	0.8239
1	IAT	0.7353	0.7318	0.7353	0.7328	0.9194	0.9226





The accuracy with IAT statistics (73.5%) was 14.5% higher than the accuracy without IAT statistics (59%). The F1 score also improved by 0.16 (0.73 vs 0.57), and the AUC improved by 0.1 (around 0.92 vs 0.82 for both one-vs-rest AUC and one-vs-one AUC). All of these scores are significant improvements.

The confusion matrix shows that while even the IAT condition struggled with classifying some difficult cases (namely distinguishing between Netflix and YouTube traffic), the IAT condition had significantly fewer misses and significantly more successful classifications (a couple hundred).

Lastly, the top features analysis shows that nearly all of the IAT stats features were determined as more important than all of the NetML STATS features, showcasing the value of including timing-based statistics in addition to standard packet-size statistics. Interestingly, the most important feature by a long shot was the minimum IAT. This suggests that the model classifies primarily based on the baseline round trip time to the nearest server or the specific burst speed of the application's connection attempts. However, the fact that the 3rd quartile IATs are the next most significant feature suggests that the server's processing time was a key

differentiator. This finding suggests that the bidirectional communications provided valuable information that helped with classification.

Overall, these results confirm my hypothesis that IAT statistics can significantly improve model performance! These results suggest that adding IAT statistics to NetML would be a huge benefit, as it could increase model performance substantially for free as we see here.